**Полный конвейер создания аватаров с нейросетью Trellis (генерация 3D), Blender, Adobe Fuse и Python**

Этот workflow позволяет автоматизировать создание, доработку и интеграцию 3D-аватаров в игру или сервис, используя **Trellis (нейросеть для генерации 3D)**, **Adobe Fuse**, **Blender** и **Python** для связки процессов.

### 1. Роли инструментов в конвейере

| Инструмент | Назначение |
|---|---|
| **Trellis (нейросеть)** | Генерация 3D-аватаров на основе входных данных (фото, текстовое описание, стиль). |
| **Adobe Fuse** | Доработка базовых моделей (одежда, прически, анатомия). |
| **Blender** | Оптимизация, риггинг, анимация, подготовка к движку. |
| **Python** | Автоматизация передачи данных между инструментами, пакетная обработка. |

### 2. Полный рабочий процесс

#### Этап 1: Генерация 3D-аватара в Trellis

- **Входные данные**:
  - Фото человека (для реалистичного аватара).
  - Текстовый запрос ("аниме-персонаж", "киберпанк-герой").
  - Параметры стиля (реализм, low-poly, мультяшный).
- **Процесс**:
  - Нейросеть Trellis создает 3D-модель в формате .obj/.fbx.
  - Возможна **первичная настройка пропорций** (рост, форма лица).

#### Этап 2: Доработка в Adobe Fuse

- **Импорт модели из Trellis** → донастройка:
  - Добавление одежды, аксессуаров.
  - Коррекция черт лица, если нейросеть неточно передала исходник.
- **Экспорт** в Blender (формат .fbx).

**Этап 3: Оптимизация и анимация в Blender**

- **Оптимизация сетки**:
  - Уменьшение полигонов (если Trellis сгенерировала слишком детализированную модель).
  - Ретопология (для корректной деформации при анимации).
- **Риггинг**:
  - Создание скелета (Armature).
  - Настройка весов (Weight Paint).
- **Анимация**:
  - Загрузка Motion Capture-данных или ручная анимация.
- **Экспорт** в игровой движок (Unity/Unreal).

**Этап 4: Автоматизация на Python**

- **Скрипты для Blender**:
  - Пакетный импорт моделей из Trellis → Fuse → Blender.
  - Автоматический риггинг по шаблону.
- **Интеграция с Trellis API**:
  - Отправка запросов на генерацию новых аватаров.
  - Парсинг выходных данных (например, из JSON-логов Trellis).

---

**3. Пример кода для интеграции**

**Python-скрипт для Blender (автоимпорт из Trellis)**

```
import bpy
import requests

# 1. Запрос к Trellis API на генерацию аватара
response = requests.post(
    ",
    json={"prompt": "мужчина 30 лет, стиль cyberpunk"},
    headers={"Authorization": "API_KEY"}
)
model_url = response.json()["download_link"]

# 2. Скачивание и импорт в Blender
```

```
import urllib.request
urllib.request.urlretrieve(model_url, "C:/temp/avatar.fbx")
bpy.ops.import_scene.fbx(filepath="C:/temp/avatar.fbx")

# 3. Автоматический риггинг (шаблонный скелет)
rig = bpy.data.armatures.new("Avatar_Rig")
obj = bpy.context.object
obj.modifiers.new("Armature", 'ARMATURE')
obj.modifiers["Armature"].object = rig
```

**Python-скрипт для Adobe Fuse (экспорт в Blender)**

*(Через CLI или AutoHotkey, если Fuse не имеет API)*

```
import subprocess
subprocess.run(["fuse-cli", "--input", "avatar.fbx", "--export",
"blender_ready.fbx"])
```

**4. Проблемы и решения**

| **Проблема** | **Решение** |
|---|---|
| **Trellis генерирует "сырые" модели** | Доработка в Fuse/Blender + Python-скрипты для автофиксов (например, сглаживание артефактов). |
| **Несовместимость форматов** | Конвертация через Blender или онлайн-сервисы (например, **Aspose.3D**). |
| **Долгая обработка** | Параллельный запуск генерации в Trellis для нескольких аватаров. |
| **Нет API у Fuse** | Использование эмуляции клавиш (**PyAutoGUI**) для автоматизации. |

**6. Итог**

Такой конвейер (**Trellis → Fuse → Blender → Python**) позволяет:
✅ **Быстро генерировать уникальных аватаров** (нейросеть + кастомизация).
✅ **Автоматизировать рутинные задачи** (риггинг, экспорт).
✅ **Гибко настраивать стиль** под проект (реализм, стилизация).

Для старта достаточно:

1. Настроить API Trellis.

2. Написать Python-скрипты для передачи данных.
3. Создать шаблоны в Blender для мгновенного риггинга.